מטלת מנחה (ממ"ן) 16 – מבוא לאבטחת המרחב המקוון

מגיש: דביר חכם פור.

מגיש: בר טופליאן.

$GROUP\ SEED = 25977022$

תוכן עניינים

מבוא 5
המערכת 5
שרת האימות 5
נקודת קצה $/register$ 5
נקודת קצה $/login$ 5
נקודת קצה $/login_totp$ 6
נקודת קצה $/admin/get_captcha_token$ 6
נקודת קצה $/captcha_verify$ 6
מסד הנתונים 6
משתמשי דמה 7
פונקציית הגיבוב 8
שיטת $sha-256$ 8
שיטת $bcrypt$ 8
שיטת $argon2id$ 8
הגנות המערכת 9
שימוש ב-$pepper$ 9
הגנת $rate-limit$ 9
הגנת $lockout$ 9
הגנת $captcha$ 10
הגנת $totp$ 10
התוקף 10
מבנה המערכת 11
מודל שכבת היישום $application$ 11
קובץ __init__.py 11
קובץ database.py 11
קובץ hash.py 11
קובץ http_status.py 11
קובץ logger.py 11
קובץ responses.py 11
קובץ server.py 12
מודל הקונפיגורציה $configuration$ 12
קובץ __init__.py 12
קובץ config.json 12
קובץ config.py 13
קובץ users.py 13

מודל ההגנה $\boldsymbol{protection}$ 13
קובץ __init__.py 13
קובץ rate_limit.py 13
קובץ lockout.py 13
קובץ captcha.py 13
קובץ totp.py 13
קובץ protection.py 13
מודל ההתקפה $\boldsymbol{attack}$ 13
קובץ __init__.py 13
קובץ brute_force.py 13
מודל הניסויים $\boldsymbol{experiments}$ 14
קובץ __init__.py 14
קובץ experiment_runner.py 14
קובץ experiment_handler.py 14
קובץ experiments_generator.py 14
תיקיית $\boldsymbol{logs}$ 14
תיקיית $\boldsymbol{plots}$ 15
קובץ experiments.json 15
תת מודל $\boldsymbol{extensive_experiments}$ 15
קובץ extensive_experiments.py 15
קובץ extensive_experiments_generator.py 16
מודל הסטטיסטיקות $\boldsymbol{statistics}$ 16
קובץ __init__.py 16
קובץ statistics.py 16
מודל הבדיקות $\boldsymbol{tests}$ 16
שאר הקבצים 16
מודל התקיפה 17
רעיון והערות מימוש 17
אופי יצירת משתמשי הדמה 17
מדוע בחרנו לממש את התקפת ה-$\boldsymbol{Brute-Force}$ בלבד 17
מימוש ההתקפה 19
ניסויים 20
מהלך הניסויים 20
רישום המידע 20
קובץ $\boldsymbol{database.db}$ 20
קובץ $\boldsymbol{users.json}$ 20

קובץ $register.log$ 21
קובץ $attempts.log$ 21
קובץ $experiments.log$ 22
יצירת מדגם הניסויים 22
ללא הגנות פעילות 23
הגנת $rate\ limit$ פעילה 23
הגנת $lockout$ פעילה 23
הגנת $captcha$ פעילה 23
הגנת $totp$ פעילה 24
הגנות $totp$ ו-$captcha$ פעילות 24
הגנות $totp$ ו-$lockout$ פעילות 24
הגנות $rate\ limit$ ו-$captcha$ פעילות 24
כלל ההגנות פעילות 24
מרחב המדגם המלא 25
תוצאות 26
גרף המתאר את זמן ה-$latency$ הממוצע בכל ניסוי 26
גרף המתאר את שכיחות כל הודעת סיבת סיום ההתקפה 26
גרף המתאר את מספר הניסיונות עבור כל ניסוי בו ההתקפה הצליחה 27
ניתוח 28
שיטת הגיבוב 28
השפעת ה-$pepper$ 28
חוזק הסיסמה 29
גורם עצירת המתקפה 30
מסקנות חשובות 31
הרחבות עתידיות 32
ניסויים 32
הגנות 32
מקורות 33

מבוא

מסמך זה בא להציג ולתאר מערכת אבטחה ואת התובנות העולות מהתקפתה.
בקורס זה התבקשנו לממש מערכת אימות המבוססת סיסמאות, לתקוף אותה ולהפיק מידע חשוב ואיכותי על עמידותן של מנגנוני אימות שונים, התקפות והגנות שונות ולהדגים כיצד שילובי הגנות ואופי מנגנון האימות משפיעים על הקלות או רמת החוזק של המערכת כנגד התקפות.

התייחסנו למטלה זו במלוא הכבוד הנדרש, הקפדנו לא "לעגל" פינות ולדאוג שהמערכת לא רק תעבוד למען המטרות המבוקשות, אלא שיהיה ניתן להוסיף הרחבות בעתיד בקלות, למזער מגע אדם בשלל חלקי התוכנית (בעיקר בבדיקות והרצת הניסויים). נתאר בפירוט יתר בהמשך המסמך לגבי האפשרויות השונות שניתנות לשינוי עבור חלקי הניסוי ולגבי המערכת, השקענו זמן רב על מנת שהמערכת תהיה כמה שיותר קלה לשינוי והטמעה ובפרט ברורה וקריאה.

המערכת

מערכת אימות זו, הינה מימוש בשפת $python$ של שרת אימות התומך במספר מנגנוני הגנה ובמספר שיטות גיבוב שונות. חלקי המערכת שונים ולכל אחד ייעוד ייחודי והצדקה לקיומו, אך החלק המרכזי במערכת הינו השרת.

במטלה זו, שרת האימות מדמה שרת אמיתי שנועד המאפשר רישום וכניסה של משתמשים.
על מנת לאשר כניסה של משתמש למערכת, נלקחים בחשבון גורמים רבים ובהתאם להגנות הפועלות, מתבצעות בדיקות והפעלת מנגנוני הגנה מסוימים.

שרת האימות

המערכת מבוסס על פלטפורמת $flask$ המאפשרת הקמת שרת בקלות. פלטפורמה זו קלה לשימוש וחוסכת למתכנת עבודה רבה.

בשרת מוגדרות כמה נקודות קצה, לכל אחת תפקיד אחר ואינן תלויות זו בזו:

נקודת קצה $/register$

נקודת קצה זו מאפשרת רישום של משתמש חדש למערכת. המשתמש יזין בבקשתו את שם המשתמש והסיסמה אותם הוא רוצה לרשום למערכת באופן הבא:

```
curl -X POST http://127.0.0.1:5000/register ^
  -H "Content-Type: application/json" ^
  -d "{\"username\":\"alice\",\"password\":\"a2b25\"}"
```

ובתגובה יחזיר השרת סטטוס בהתאם לבקשה.
הסטטוס יקבל ערכים שונים על פי תוצאות שונות (שגיאת כפילויות, פרטים חסרים, סיסמה לא חוקית, הרשמה בהצלחה).

נקודת קצה $/login$

נקודת קצה זו מאפשרת כניסה למערכת, למעשה מדובר בניסיון כניסה מפני שלא כל בקשה שכזו תאושר (ובחלק מהמקרים אפילו שפרטי המשתמש והסיסמה נכונים). המשתמש יזין בבקשתו את שם המשתמש והסיסמה, למשל:

```
curl -X POST http://127.0.0.1:5000/login ^
  -H "Content-Type: application/json" ^
  -d "{\"username\":\"alice\",\"password\":\"a3b25\"}"
```

והשרת יחזיר תגובות שונות בהתאם לתוצאה (משתמש לא קיים, פרטים חסרים, מספר ניסיונות רב מידי וכיוצא בזה).

נקודת קצה $/login_totp$

נקודת קצה זו מאפשרת למשתמש להזין קוד $totp$ (קוד זמני הנגזר מסוד משותף) במידה ופרטי כניסתו למערכת נכונים אך מופעלת הגנה זו. אחרי בקשת המשתמש לנקודת הקצה הקודמת למען כניסה למערכת, במידה והגנה זו מופעלת, יחזיר השרת תגובה המבקשת הזנה של קוד זמני זה. המשתמש יזין את שם המשתמש והקוד הזמני באופן הבא:

```
curl -X POST http://127.0.0.1:5000/login_totp ^
  -H "Content-Type: application/json" ^
  -d "{\"username\":\"alice\",\"code\":\"729429349264\"}"
```

השרת יחזיר תגובה בהתאם להצלחת/כשלון אימות הקוד הזמני (או האם אחד הפרטים לא נכונים וכיוצא בזה).

נקודת קצה $/admin/get_captcha_token$

נקודת קצה זו נועדה בעיקר למען בדיקות מערכת עבור הגנת ה-$captcha$, מנגנון זו בדומה לזו המדוברת עבור נקודת הקצה הקודמת, יופעל במידה והגנה זו מופעלת. המשתמש/מנהל יזין בבקשתו את ה-$GROUP_SEED$ (קבוע המורכב מפעולת XOR של שני מספרי תעודות הזהות של כותבי המסמך), באופן הבא:

```
curl -G "http://127.0.0.1:5000/admin/get_captcha_token" ^
  --data-urlencode "group_seed=25977022"
```

ובמידה וה-$seed$ חוקי השרת יחזיר קוד $captcha$ כמתבקש.

נקודת קצה $/captcha_verify$

נקודת קצה זו בדומה לקודמת, נועדה בכדי לשלוח לשרת את ערך קוד $captcha$ שהתקבל. המשתמש/מנהל יזין קוד זה באופן הבא:

```
curl -X POST "http://127.0.0.1:5000/login" ^
  -H "Content-Type: application/json" ^
  -d "{\"username\": \"alice\", \"password\": \"a2b25\", \"captcha_token\":
\"d546bf7ef97c0464a7a18cefcabbdc3ef6597cd21ef848e365538f1f4bf91b7\"}"
```

והשרת יחזיר תגובה על פי נכונות המידע שהתקבל.

מסד הנתונים

עבור הסימולציה הקיימת של שרת האימות נוצר מסד נתונים אליו ישמרו פרטי המשתמשים וערכי הגיבוב של הסיסמאות.

השתמשנו בספרייה $sqlite3$ המאפשרת הקמה של בסיס נתונים בקלות יחסית וללא מורכבות בלוגיקת מבני נתונים. במסד הנתונים מוגדרת טבלה אחת ובכל רשומה מאוחסנים שם המשתמש וערך הגיבוב של הסיסמה. הגדרת מסד הנתונים נראה כך:

```
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT UNIQUE,
  hashed_password TEXT
```

משתמשי דמה

לאחר יצירת מסד הנתונים, המערכת תיצור כשלושים משתמשי דמה ותוסיף אותם למערכת. הוספה של משתמשים אלו יהיו ישירות אל מסד הנתונים ולא יעברו דרך נקודות הקצה של השרת.

המשתמשים מחולקים לשלוש קבוצות על פי סיווג חוזקתם:

- משתמש חלש – אורך סיסמה בין 4 ל-5 תווים מתוך סט התווים הבא: ab0123456.
  סט תווים מצומצם זה בשילוב עם אורך סיסמה קצר, מאפשר פיצוח הסיסמה באמצעות התקפה ממצה עבור חלק מהמקרים. כך ניתן לקבל מבט כולל על התנהגות המערכת בעת התקפה נכונה.

  סך הכל מרחב מדגם של כ-65610 ($9^4 + 9^5$) סיסמאות חוקיות.

- משתמש בינוני – אורך סיסמה בין 8 ל-12 תווים הכולל תווים אפשריים מסט התווים המכיל אותיות "נמוכות", "גבוהות" וספרות.

  סך הכל מרחב מדגם של כ-$3,379,998,194,577,880,416$ סיסמאות (חישוב טריוויאלי).

- משתמש חזק – אורך סיסמה בין 14 ל-20 תווים הכולל תווים אפשריים מסט התווים המכיל אותיות "נמוכות", "גבוהות", סימני פיסוק ותווים מיוחדים וספרות.

  סך הכל מרחב מדגם של כ-$3,180,294,872,769,072,889,939,712$ סיסמאות.

נקודות חשובות עבור משתמשי הדמה:

- סיסמתו של משתמש דמה אקראי (אחד) המסווג לחוזקה הבינונית תמיד יהיה שווה לערך ה-$GROUP_SEED$.
- לפחות משתמש אחד מכל חוזקה יקבל ערך $totp_secret$, כדי שיהיה ניתן לבצע התקפה עם הגנה זו על פני משתמשים מכלל הסיווגים.

ערכי משתמשי הדמה ישמרו בקובץ $users.json$ אליו יתווספו שאר פרטי המשתמשים החדשים לאחר ביצוע בקשה מוצלחת לנקודת הקצה הרלוונטית להרשמת משתמשים.

ערכים נבחרים מקובץ שכזה:


```
{
  "username": "weak_user_1",
  "password": "235b",
  "strength": "weak",
  "totp_secret": "42OC7KJYALTUABAITVXFO2DKBSBNCTOM"
},
{
  "username": "medium_user_1",
  "password": "oDna4wgk5mI",
  "strength": "medium",
  "totp_secret": null
},
{
  "username": "strong_user_2",
  "password": "NRZEoaSe#6*7\\*",
  "strength": "strong",
  "totp_secret": "N623SRBWVCD2SF5C4IQ6ISIMVZB7H6I5"
},
```

פונקציית הגיבוב

כאשר רוצים לממש מערכת אבטחה ואף הבסיסית ו-"הגסה" ביותר, חשוב לדעת שחוזק המערכת נמדד על פי החולייה החלשה ביותר בשרשרת, וגם מנגנון אבטחה משומן, בעל עשרות הגנות מתקדמות, הצפנות פוסט-קוואנטיות ואפס סובלנות לניסיונות אימות, לא יחשב כחזק אילו מסד הנתונים שלו יהיה פרוץ לכל.

בהקשר למטלה זו, התבקשנו לבנות סביבת מערכת פשוטה יחסית אך שתקיים הגיון ממשי בדומה למערכות אבטחה *אמיתיות*. הדרך הפשוטה ביותר והראשונה למימוש, היא שימוש בפונקציית גיבוב עבור הסיסמאות. בדומה לנאמר מקודם, חוזק המערכת לא נמדד על פי החוזקות אלא על פי החולשות ואחסון סיסמאות בצורה גלויה במסד הנתונים היא לא פחות מחולשה בעלת קנה מידה הרסני. תוקף שיצליח לפגוע במסד הנתונים ולשים ידו על קבצי מסד הנתונים הגולמיים, יקבל באותה הזדמנות את פרטי כל המשתמשים ואת הסיסמאות שלהם, ללא צורך בהתקפה נוספת.

מטרתן של כלל פונקציות הגיבוב היא לקבל קלט באורך חופשי (במקרה זה, סיסמה) ולהמיר אותה לפלט באורך קבוע. עקרון מרכזי בפונקציות גיבוב (חזקות קריפטוגרפית) הוא בהינתן פלט כלשהו לא יהיה ניתן לשחזר את הקלט שבאמצעותו הוא חושב. עקרון זה בנוסף עם שאר העקרונות המבטיחים שלא יהיה ניתן למצוא התנגשויות, תרמו לביסוס פונקציות אלו בתוך מנגנוני אימות והצפנה[1].

כדי למנוע מצב שכזה בו הסיסמאות שמורות כטקסט גלוי במסד הנתונים, נשמור במקום זאת ערך מגובב של הסיסמה באחת משלוש שיטות הגיבוב הבאות:

שיטת $sha-256$

פונקציית $sha-256$ היא פונקציית גיבוב קריפטוגרפית שמייצרת פלט בגודל 256 סיביות. היא חד-כיוונית, כלומר ניתן לבצע גיבוב לערך כלשהו אך לא ניתן למצוא את ערך המקור של תוצאת גיבוב. פונקציה זו נחשבת מהירה מאוד (נתייחס למהירותה בחלק ניתוח הנתונים בהמשך), גם במבנה ברירת המחדל שלה וגם כאשר נעשה שימוש ב-$salt$ (ערך לא בהכרח סודי וייחודי למשתמש הנוסף לסיסמה לפני הגיבוב), זמן החישוב לא עולה משמעותית, בניגוד לשיטות הבאות עליהן נרחיב בהמשך.

במערכת המתוארת ניתן לקבוע את אורך ה-$salt$ כאחד מהערכים הניתנים לשינוי בקובץ ההגדרות (`config.json`) ובמימוש פונקציה זו ערך ה-$salt$ מתווסף לפני הערך המגובב במחרוזת הנשמרת במסד הנתונים.

שיטת $bcrypt$

שיטה זו בניגוד לפונקציות גיבוב נפוצות הנחשבות "מהירות" או שאינן משנות את זמן פעולתן ובפרט מפונקציית $sha-256$, פונקציית $bcrypt$ מסווגת כפונקציית גיבוב "איטית". סיווג זה נקבע על ידי התכונה המרכזית המאפיינת את פעילות הפונקציה: לאורך זמן ועל ידי פרמטרים הניתנים לשינוי, מספר האיטרציות הדרושות בשלב הגיבוב יכול לגדול ובכך להאריך את זמן החישוב הדרוש. תכונה זו יעילה וחיונית ביותר בכדי להתמודד ולהיות חסינה במידה רבה לסוגי מתקפות שונות ובפרט מתקפות $Brute-Force$ אפילו אם כוח חישוב מואץ ומשאבי עיבוד רבים[1].

שיטת $argon2id$

שיטה זו נחשבת לחזקה ביותר מבין הפונקציות שלעיל, היא מודרנית ומומלצת לאחסון סיסמאות. היא משלבת עמידות נגד התקפות ערוץ-צדדי*, מאפשרת לשלוט בזמן החישוב, בזיכרון ובמקבילות. כיום היא נחשבת לפתרון הבטוח והגמיש ביותר לאחסון סיסמאות[3].

---

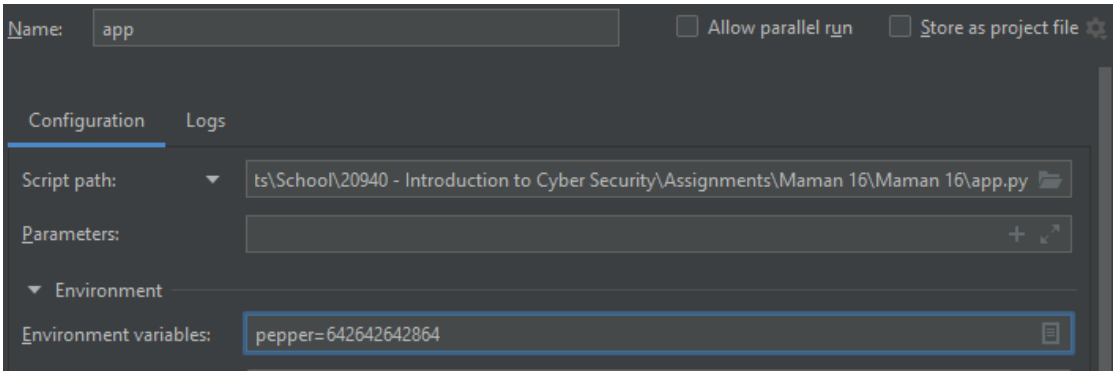
* היא התקפה קריפטוגרפית המנצלת מידע מהיישום הפיזי של מערכת ההצפנה ולא מהאלגוריתם עצמו. היא נעשית באמצעות ניתוח פרטים כמו זמני עיבוד, צריכת אנרגיה, קרינה אלקטרומגנטית, רעשים, והשתיירות מגנטית על מדיה, וכן הזרקת שגיאות. זאת עקב זיהוי וניצול מגבלות טכניות או כשלים ביישום עלולים לחשוף מידע שיסכן את ביטחון המערכת[2].

הגנות המערכת

שימוש ב-$pepper$

ערך ה-$pepper$ בדומה ל-$salt$ מתווסף לסיסמה לפני הגיבוב, אך בשונה מזאת הוא אינו נשמר במסד הנתונים, הוא אינו ייחודי, גלובאלי וסודי לכל משתמש ובמערכת זו הוא מתקבל כמשתנה סביבה.

ניתן לספק למערכת ערך שכזה בכמה דרכים, בין אם על ידי פקודה בטרמינל (למשל בחלונות: *set pepper=value*), אנחנו הזנו ערך זה דרך סביבת העבודה:



ועבור הניסויים השונים השתמשנו בערך הגלובלי בצורה הבאה בקטע הקוד:

```
BASE_PROTECTIONS = {"pepper": get_environmental_pepper()}
```

הגנת $rate-limit$

הגנה זו (יחד עם שאר ההגנות מנקודה זו) מקבלת פרמטרים המשפיעים על תפקודה מקובץ הקונפיגורציה עליו יוסבר בהמשך.

בעת בקשת כניסה לשרת במידה והגנה זו פועלת, יבדוק השרת האם מספר הכניסה הכושלים שבוצעו על ידי כתובת ה-IP של מבקש הבקשה עבר את מספר הניסיונות הכושלים המותרים. במקרה זה יחסיר השרת הודעת שגיאה המציינת כי נעשה מספר רב מידי של ניסיונות כניסה למערכת.

מספר הניסיונות המותרים (כמה "טוקנים" נשארו לכתובת) מתאפס בעת כניסה מוצלחת אך גם במידה ולא, ישנו פרמטר המגדיר את מספר הטוקנים שמתווספים (עד לגבול המוגדר) בכל שנייה.

נציין כי במתקפה שלנו ישנה אפשרות בעת קבלת חסימה שכזו להחליף את כתובת ה-IP שממנה התוקף מבקש את בקשות הכניסה ולמעשה ניתן לעקוף ככה את מנגנון זה.

הגנת $lockout$

בדומה להגנה הקודמת, כאן מתבצעת חסימה על פי שם משתמש ולאו דווקא על פי כתובת ה-IP שלו. כך גם במידה ויחליף התוקף את כתובתו, המשתמש יהיה חסום בכל זאת. המערכת מקבלת פרמטרים לכמה ניסיונות כושלים מותר לבצע, קצב חידוש האשרות, זמן החסימה הזמנית וקצב זמן החסימה.

חסימה מוחלטת של משתמש הינה צעד רציני וכהרחבה עתידית ידרוש אישור מנהל לאיפוס המשתמש (בדרך כלל כרוך ביצירת סיסמה חדשה). לכן במימוש שלנו, כאשר המכסה הראשונית של האשרות הגיעה לסופה, המערכת תחסום את המשתמש מלבצע ניסיונות כניסה למשל הזמן המוגדר בקובץ הקונפיגורציה.

לאחר שזמן זה תם, מספר האשרות החדש עבור משתמש זה יהיה חלקי לערך המקורי. קצב החלוקה נקבע גם הוא בקונפיגורציה. בנוסף, זמן המתנה עד חידוש האשרות גדל בצורה מעריכית על פי פרמטר הקצב.

לאחר שתמו כל האשרות וקצב החידוש המתעדכן גרם למספר האשרות הבא להיות אפס, השרת יבצע חסימה קבועה של המשתמש ויחזיר הודעת שגיאה בהתאמה.

במידה ומשתמש הזין פרטים נכונים, ערכי האשרות והקצבים יחזרו להיות כברירת המחדל.

נציין כי הוספנו אפשרות לעצור את המתקפה לחלוטין במידה והשרת מחזיר חסימה קבועה למשתמש אותו ירצה התוקף לתקוף. התוקף ידע להבחין בין חסימה זמנית לחסימה קבועה על פי התגובה החוזרת מהשרת.

הגנת $captcha$

כאשר מנגנון זה מופעל אחרי מספר ניסיונות כושלים (מזוהה על פי כתובת IP), יחזיר השרת הודעת בקשה לערך קוד $captcha$ זמני. המשתמש/מנהל יזין בבקשתו את ה-$GROUP_SEED$ ובמידה וה-$seed$ חוקי השרת יחזיר קוד $captcha$ כמתבקש.

לאחר הזנת קוד חוקי, המערכת תאפס את מספר הניסיונות הכושלים עבור כתובת זו על פי ערכי ברירת המחדל המוגדרים גם כן בקובץ הקונפיגורציה.

במידה והערך שהוזן אינו חוקי/לא נכון, השרת יחסום את כתובת ה-IP של התוקף. בדומה למקודם החלפת כתובות בעצם תתגבר על הגנה זו.

הגנת $totp$

כאשר הגנה זו מופעלת השרת יבקש קוד זמני הנגזר מסוד משותף ($totp_secret$) טרם אישור כניסתו של המשתמש למערכת. הגנה זו נועדה לספק מעמסה נוספת עבור התקפות ממצות, גם במקרה בו הסיסמה נכונה, אם קוד זה אינו נכון או לא תואם לחלון הזמנים והסטייה המוגדרים במערכת המשתמש יחסם לצמיתות מהמערכת.

ההיגיון בחסימה זו הוא שבמידה ותוקף הצליח לשים ידו בדרך זדונית על הסיסמה, עדיין לא יהיה יכול להיכנס למערכת כל עוד אינו יודע את הסוד המשותף. הסוד הנוצר מאומת על ידי המערכת והיא במקומה בודקת האם הקוד מתאים לחלון זמן אחר או לסטיית זמן מוגדרת, כך תוכל לאפשר כניסה גם במידה וחלון הזמנים/סטייה לא זהה לקוד הנוצר על פי חותמת זמן כלשהי. בצורה כזו יהיה ניתן להתמודד טוב יותר עם חוסר סנכרון במערכת ולמנוע חסימות שווא.

בניגוד להגנות הקודמות, בכל כניסה למערכת יידרש המשתמש להזין קוד $totp$. קוד זה יהיה שונה בכל פעם מכיוון שאינו רק זמני אלא תלוי בזמן ייצורו.

התוקף

תוקף הוא כל ישות זדונית שמטרתה לפרוץ למערכת על ידי הזנת פרטי כניסה שלא שייכים אליה. לתוקף יש ידע מוגבל על המערכת ועל המשתמשים והכלים העומדים לרשותו בסיסיים ולא מופרכים. נציין כמה הנחות שהסתמכנו עליהן עבור התוקף:

- התוקף יודע את שמות משתמשי המערכת.
- התוקף יודע את חוזק סיסמתם של כל אחד ממשתמשי המערכת.
  (ניתן לבצע מתקפה כאשר חוזק הסיסמה אינו ידוע, למעשה אקראי).
- התוקף לא יודע אילו הגנות פועלות במערכת ואינו יודע כיצד ממומשות.
- לתוקף אין גישה למסד הנתונים או פרצה לשרת המאפשרת לו התקפה שאינה מקוונת.
- התוקף יכול לשנות את כתובת ה-IP שלו ככל שירצה.
- התוקף לא יכול לייצר קוד $totp$ או $captcha$ (ניתן לבצע מתקפה כאשר התוקף יכול לייצר קודים שכאלו).

מבנה המערכת

המערכת מורכבת מחלקים רבים של מודלים ותתי מודלים. כל מודל, מחלקה, פונקציה מבצעת דבר אחד אבסולוטי ואין ריכוז סמכויות כפול.

- חלק מרכיבי המערכת נועדו לבדיקה עצמית (של כותבי המסמך).
- חלק מרכיבי המערכת נועדו להרחבות עתידיות.
- חלק מרכיבי המערכת נועדו לניסויים בקנה מידה רחב הרבה יותר מהמתבקש.

למטלה זו מצורף המערכת כולה על שלל חלקיה. לא כל המערכת כתובה באותו אופן ולא כל חלקי המערכת (גם אלו שנועדו לבדיקה עצמית) קיימים למטרת הבודק. נפרט בדיוק על המודלים הרלוונטיים ונבקש להסב תשומת לב למודלים העיקריים.

בנוסף, ישנו קובץ "ראשי" ($app.py$) בו ניתן להריץ את השרת על גבי הרשת. לא נשתמש בו בניסוי והוא קיים כדי לאפשר הרצה רגילה של השרת (כמו ששרת אמיתי רץ ברשת).
את כלל הבקשות אל השרת במודלים של המערכת נבצע באמצעות "לקוח בדיקה" ($test_client$) של פלטפורמת ה-$flask$ המדמה הרצה אמיתית של השרת מבלי הרצתו ברשת.

פירוט המודלים הבאים מובא על פי סדר חשיבות ולאו דווקא מבנה היררכי בקבצי המערכת.

מודל שכבת היישום $application$

מודל זה אחראי על הקמתו, אתחולו, ניהולו ופעולתו של שרת האימות. במודל זה הקבצים הבאים:

קובץ __init__.py

אחראי להוציא את המחלקות, מבני הנתונים והפונקציות הרלוונטיות מתוך המודל החוצה.

```python
from .hash import get_hashing_function, get_hashing_verification_function
from .logger import Logger
from .http_status import HTTPStatus, CAPTCHA_REQUIRED, TOTP_REQUIRED, ACCOUNT_BLOCKED, ACCOUNT_TEMPORARILY_BLOCKED
from .database import Database
from .server import AuthServer
from .responses import ResponseHandler
```

קובץ database.py

אחראי על הגדרה של מסד הנתונים, הכנסת רשומה חדשה, שליפה של ערך הגיבוב של משתמש לצורך השוואה ובדיקת קיימות שם משתמש במערך.

קובץ hash.py

מספק את פונקציית הגיבוב ופונקציית האימות עבור כל שיטת גיבוב המוזכרת למעלה.
בהתאם לסוג שיטת הגיבוב והפרמטרים המתאימים אליה, תיצור המחלקה פונקציות גיבוב ואימות.

קובץ http_status.py

מכיל מבנה נתונים $enum$ המקשר בין קוד שגיאה $HTTP$ לבין ערכו המספרי.
בנוסף מכיל מחרוזות להודעות שגיאה בהן יכול התוקף להשתמש כדי לקבוע את המשך מתקפתו.

קובץ logger.py

מכיל פונקציות המאפשרות הוספת מידע לקבצי לוג (log). שרת האימות (בעקיפין) משתמש בשיטות אלו כדי לרשום כל יצירת משתמש חדש ($register.log$), ניסיון כניסה ($attempts.log$) ותוצאות ניסוי (נקבע על פי הניסוי).

קובץ responses.py

מכיל פונקציות המבצעות רשימה ללוג והחזרה של סטטוס $HTTP$. למעשה אחראי על תגובות השרת כלפי המשתמש.

קובץ server.py

לב ליבת המערכת, אחראית על הקמת השרת, הגדרת נקודות הקצה, יצירת משתמשי דמה.
מתפעל את השרת בעת בקשת יצירת משתמש, כניסה, מבצע בדיקת הגנות ודואג לנהל את כל מערכת האימות.
באמצעות שימוש בלוגיקה מכל מודל בהתאמה, במחלקה זו נשלטת כל תפעול המערכת.

מודל הקונפיגורציה $configuration$

מודל זה אחראי להתמודד עם כל הוריאנטים של פרמטרי השרת (שיטת גיבוב, פרמטרים ליצירת סיסמאות, הגנות פעילות). ולאפשר שליפת מידע שבאמצעותו יוגדרו רוב חלקי המערכת.

קובץ __init__.py

אחראי להוציא את המחלקות והפונקציות הרלוונטיות מתוך המודל החוצה.

```python
from .config import config
from .users import DummyMembersManager

GROUP_SEED = config.group_seed

hash_mode = config.hash_mode
get_hash_params = config.get_hash_params
get_environmental_pepper = config.get_environmental_pepper

protections = config.get_protection_with_params()
rate_limit_parameters = config.rate_limit_parameters
lockout_parameters = config.lockout_parameters
captcha_parameters = config.captcha_parameters
totp_parameters = config.totp_parameters

get_password_params = config.get_password_params

generate_password = DummyMembersManager.generate_password
```

קובץ config.json

קובץ זה מכיל את כל סוגי הפרמטרים המשתנים במערכת ואת ערכי ברירת המחדל שלהם.
בעזרת קובץ זה ניתן להגדיר כשורה כל מנגנון במערכת.

```json
{
  "hash_mode": "sha256",
  "hash_parameters": {
    "sha256": { "salt_length": 16 },
    "bcrypt": { "cost": 12 },
    "argon2id": { "time_cost": 1, "memory_cost": 65536, "parallelism": 1 }
  },
  "group_seed": "25977022",
  "protections": {
    "pepper_enabled": true,
    "rate_limit_enabled": false,
    "lockout_enabled": false,
    "captcha_enabled": false,
    "totp_enabled": false
  },
  "protections_parameters": {
    "rate_limit_parameters": { "tokens": 4, "refill_rate_tps": 1 },
    "lockout_parameters": { "tokens": 9, "token_rate": 5, "duration_rate": 1, "duration_mm": 0.1 },
    "captcha_parameters": { "tokens": 3, "time_to_live_ss": 5 },
    "totp_parameters": { "number_of_users": 5, "tokens": 3, "time_step_ss": 30, "tolerance_ss": 30 }
  },
  "password_parameters": {
    "weak": { "min_length": 4, "max_length": 5, "charset": "0123456ab" },
    "medium": { "min_length": 8, "max_length": 12, "include_upper": true, "include_digits": true },
    "strong": { "min_length": 14, "max_length": 20, "include_upper": true, "include_digits": true, "include_punctuation": true }
  }
```

קובץ config.py

אחראי לשליפת כל סוג פרמטר מקובץ הקונפיגורציה. הנגשת הערכים בצורה קלה יותר, מענה למשיכת ערך ה-$pepper$ מהסביבה ויצירת מבנה נתונים המכיל את כל ההגנות הפעילות ואת ערכי הפרמטרים שלהם לטובת שאר חלקי המערכת.

קובץ users.py

אחראי על יצירת משתמשי דמה, הוספת משתמשים חדשים וכן הספקת פונקציות להחזרת שם משתמש אקראי לטובת המתקפה.

מודל ההגנה $protection$

מודל זה מרכז את כלל מחלקות ההגנה השונות ואת תפקודן.

קובץ __init__.py

אחראי להוציא את המחלקות הרלוונטיות מתוך המודל החוצה.

```
from .protection import ProtectionHandler
```

קובץ rate_limit.py

אחראי על פעולת מנגנון הגבלת הבקשות ומאפשר בדיקת אשרת בקשה לפי כתובת IP.

קובץ lockout.py

אחראי על פעולת מנגנון הנעילה, מכיל בתוכו לוגיקה לבדיקת אשרת בקשה לפי שם משתמש, נעילת משתמש ואיפוס קצב הגבלת המשתמש.

קובץ captcha.py

מכיל בתוכו פונקציות לבדיקת צורך באימות על ידי $captcha$, נעילת תוקף על פי כתובת IP, יצירה ואימות קודי $captcha$ זמניים ורשימת ניסיונות כניסה כושלים על פי כתובת IP.

קובץ totp.py

מכיל בתוכו פונקציות ליצירת קוד $totp$ זמני ואימות קודים שכאלו תוך כדי גמישות/נוקשות כלפי רוחב חלון וסטיית זמן וחסימת שם משתמש.

קובץ protection.py

מנהל את כל יחידות ההגנה ומספק מעטפת קלה, נוחה יותר ועמוסה פחות לשימוש על ידי שרת האימות.

מודל ההתקפה $attack$

מודל זה אחראי על התקפת המערכת, על ידי ביצוע מתקפה ממצה על מרחב (חלקי) הסיסמאות. במודל זה מוגדרת הלוגיקה האחראית על התקיפה (על אופי המודל עצמו נפרט בפרק הבא).

קובץ __init__.py

אחראי להוציא את המחלקות הרלוונטיות מתוך המודל החוצה.

```
from .brute_force import BruteForceAttack
```

קובץ brute_force.py

אחראי לבצע את המתקפה עצמה, בעזרת פונקציית ה-$attack$ המוגדרת בו, יבצע התוקף התקפה של שרת האימות. על פי הפרמטרים שיקבל, יהיה יכול לשלוט במהלך ההתקפה (מתי והאם לעצור, האם לשנות כתובת IP, וכיוצא בזה).

מודל הניסויים *experiments*

מודל זה אחראי לבצע את הניסויים עצמם, לאסוף את המידע הרלוונטי ולסווג אותו על פי סוגי מתקפות/שיטות גיבוב בכדי שיהיה ניתן להפיק סטטיסטיקות ממנו.

קובץ __init__.py

אחראי להוציא את המחלקות הרלוונטיות מתוך המודל החוצה.

```
from .extensive_experiments.extensive_experiments import build_groups, generate_experiment_configs
from .experiment_runner import ExperimentRunner
```

קובץ experiment_runner.py

אחראי לאתחל את תיקיות הפלט (עבור קבצי הלוג ועבור הגרפים), לקבל את רשימת הניסויים, להריץ את כלל הניסויים (ניתן להריץ על פי קטגוריה או את כולם) ולקרוא למחלקת הסטטיסטיקות על מנת להפיק גרפים מתוצאות הניסויים.

קובץ experiment_handler.py

אחראי לנהל כל מהלך ניסוי:

1. אתחול שרת האימות על פי סוג הגיבוב/ההגנות ושאר הפרמטרים המוגדרים לכל ניסוי.
2. אתחול מחלקת ההתקפה בהתאם להגדרות שנקבעו לכל ניסוי.
3. הרצת המתקפה ומדידת זמן ההתקפה.
4. שמירת קובץ הלוג של כל ניסוי בתיקייה מתאימה על פי הקטגוריה (אופציונאלי)*.
5. איפוס, סגירה ומחיקת הקבצים שאינם רלוונטיים וסגירת שרת האימות הנוכחי.

למעשה, קובץ זה אוגד בתוכו את הלוגיקה עבור מימוש והרצת כל ניסוי. בניגוד לקובץ שתואר קודם לכן, קובץ זה מתמקד יותר בהגדרות והאתחולים הנדרשים על מנת לבצע כל ניסוי מאשר הרצתו באופן כללי.

קובץ experiments_generator.py

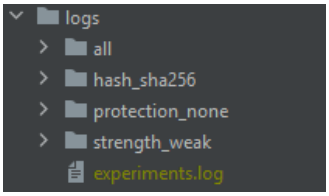
אחראי על יצירת מדגם הניסויים המצומצם עבור ההתקפות. קובץ זה מגדיר כ-27 ניסויים שונים, כל אחד שונה ממשנהו בחוזק הסיסמה/ההגנות הפועלות/דגלי ההתקפה ושאר הפרמטרים המייחדים כל ניסוי ועוזרים ליצור מרחב מדגם מצומצם אך חיוני בהחלט.

בהמשך המסמך נתאר את סוגי הניסויים שבחרנו לממש, את ערכי ההגנות, דגלי ההתקפה ושאר הערכים המשתנים.

תיקיית *logs*

מכילה את קבצי הלוג הסופיים המאגדים את כלל תוצאות הניסויים (באופן כללי רק קובץ אחד, `experiments.log`). ובמידה ונדליק דגל רלוונטי במחלקת הרצת הניסויים, ישמרו בתיקייה זו קובצי `attempts.log` עבור כל ניסוי.

למשל (במידה ודגל זה יופעל):



* כפי שיפורט בהרחבה בפרק העוסק בניסויים, הרצת הניסויים מתבצעת בסביבה זמנית וכלל הקבצים הנוצרים בזמן כל ניסוי (`register.log, attempts.log` ו-`database.db`) ימחקו אחרי הרצת הניסוי. ניתן לשמור את קובץ `attempts.log` (ניתן לשנות מעט את הקוד כדי לשמור את שאר הקבצים, או לא לבצע את הניסויים בסביבה זמנית) בכדי לשמור את יתר הקבצים.

תיקיית *plots*

מכילה קבצי תמונות ($.png$) עבור כל גרף שנוצר בעקבות תוצאות הרצת הניסויים.

קובץ experiments.json

קובץ זה מגדיר את כלל הפרמטרים וערכי ברירת המחדל שלהם עבור הגדרת ניסוי.
בדומה לקובץ הקונפיגורציה הכללי מופיעים ערכי שיטות הגיבוב והפרמטרים עבורם, אך בשונה מקובץ הקונפיגורציה הכללי, ניתן להגדיר כמה ערכים לכל שדה בפרמטרי ההגנות.

כפי שנסביר בצורה רחבה יותר במעט בהמשך, הרחבה עתידית אפשרית היא הרצה של סט ניסויים רחב משמעותית מסט הניסויים המדגמי אותו נציג בחלק הניסוי. על ידי העמסת ערכים שונים יהיה ניתן להרחיב את מספר הניסויים ולדגום נתונים איכותיים עוד יותר.

ניתן למשל לשנות את ערך עלות פונקציית הגיבוב $bcrypt$ מ-12 ל-24 וכתוצאה מכך להגדיל את זמן הריצה הדרוש למתקפה (עקב זמן פעולת גיבוב איטית יותר).

```
"hash_options": {
  "sha256": { "salt_length": 16 },
  "bcrypt": { "cost": 12 },
  "argon2id": { "time_cost": 1, "memory_cost": 65536, "parallelism": 1 }
},

"protections": {
  "rate_limit": {"tokens": [3, 5], "refill_rate_tps": [1, 2]},
  "captcha": {"tokens": [3], "time_to_live": [60, 120]},
  "lockout": {"tokens": [3], "token_rate": [2], "duration_rate": [1], "duration_mm": [0, 5]},
  "totp": {"number_of_users": [5, 10], "tokens": [3], "time_step_ss": [30], "tolerance_ss": [30]}
},

"attack_options": {
  "password_strength": ["weak", "medium", "strong"],
  "max_attempts": [50000],
  "delay_ss": [0, 0.1, 1],
  "alternate_strength_enabled": [true, false],
  "ip_switch_enabled": [true, false],
  "lockout_stop_enabled": [true, false],
  "captcha_token_enabled": [true, false]
}
```

בנוסף, ניתן להבחין כי ישנה אפשרות לשנות ערכים לאפשרויות המתקפה. בניגוד לסט הניסויים המדגמי שאותו בחרנו, ניתן לראות את התנהגות המערכת כאשר התוקף ישנה את כתובת ה-IP שלו על ידי הדגל $ip_switch_enabled$ וכך למעשה למנוע חסימה (ולהתגבר) ממנגנון ההגנה $rate\ limit$ למשל.

בקובץ זה ניתן לשנות ערך מספר הניסיונות המרבי בכל התקפה, כך ניתן למשל את ערך זה להיות גדול ממרחב המפתחות ולהגדיל את סיכויי הצלחת המתקפה[†].

תת מודל *extensive_experiments*

תת מודל זה הינו הרחבה עתידית של מרחב מדגם ניסויים רחב ומקיף הרבה יותר מהמדגם הנוכחי.

קובץ extensive_experiments.py

קובץ זה אחראי על יצירה של מרחב המדגם הרחב, הוא יוצר תצורות ניסוי כוללות יותר, עבור מספר רב יותר של צירופי הגנות, דגלי התקפה, פרמטרי גיבוב וכיוצא בזה.

למרות שמדובר בקובץ לשימוש עתידי ואין בכוונתנו להשתמש בו, שהרצתו חוקית ותעבוד וניתן להרחיב את מדגם הניסויים עוד יותר על ידי הוספת לוגיקה בקובץ זה.

[†] נציין כי למרות שעבור 50000 הניסיונות הנוכחי שקטן באופן ניכר מ-36864 המפתחות האפשריים עבור סיסמאות חלשות, פונקציית יצירת הסיסמאות הנוכחית מייצרת מפתחות אקראיים ולא מבצעת יצירה ממצה של כל התמורות. לפיכך גם במודל הקיים ערך הניסיונות לא מבטיח הצלחה וודאית לכל התקפה (ללא הגנות) של משתמש בעל סיסמה חלשה.

קובץ extensive_experiments_generator.py

בדומה לקובץ הקודם, קובץ זה מייצר מדגם ניסויים רחב יותר ושימושו עתידי.
בשונה מהקובץ הקודם, הוא פחות רחב וכולל מרחב מצומצם יותר (אך עדיין רחב יותר מהמדגם הנוכחי).

מודל הסטטיסטיקות $statistics$

אחראי להפקת גרפים, ניתוח מידע וחישוב סטטיסטי על פי תוצאות הניסויים.

קובץ __init__.py

אחראי להוציא את המחלקות הרלוונטיות מתוך המודל החוצה.

```
from .statistics import StatisticsPlotter
```

קובץ statistics.py

מכיל בתוכו כמה פונקציות ליצירת גרפים שונים (זמן ריצה ממוצע לכל ניסוי, ניתוח על פי תוצאות הניסויים ומספר הניסיונות שנדרשו בכל ניסוי בו המתקפה הצליחה).

מודל הבדיקות $tests$

מודל זה אחראי לרכז את כלל בדיקות (יחידה) עבור חלקים שונים במערכת. חלק מהקבצים תואמים לגרסאות קודמות יותר של המערכת ולא בהכרח יעבדו מבלי שינוי קל.
השתמשנו בבדיקות אלו על מנת להבין את התנהגות המערכת ואת נכונות המימוש.

- $test_auth_server$ – מכיל בדיקות עבור השרת.
- $test_brute_force$ – מכיל בדיקות עבור אורכי סיסמות שונים ומימושים שונים של מחלקת התקיפה.
- $test_experiment$ – מכיל בדיקות עבור שילובי ניסויים ומנגנון יצירת הניסויים.
- $test_experiment_parameters$ – מכיל בדיקות עבור יצירת מרחבי ניסויים בתצורה חדשה יותר.
- $test_system$ – מכיל בדיקות לתפקוד השרת ובקשות אליו (גרסה חדשה יותר).
- $test_totp$ – מכיל בדיקות על תפקוד ההגנה הזו.

שאר הקבצים

כפי שציינו קודם לכן, קובץ ה-$app.py$, נועד להרצת השרת על גבי הרשת. כתוצאה מהרצה זו נוצרים ארבעה קבצים נוספים: `register.log, attempts.log users.json` ו-`database.db`.

השארנו את הקבצים האלו על מנת שתוכלו להתרשם מהמבנה הכללי שלהם, הרי כפי שציינו ונפרט בהמשך, קבצים אלו הנוצרים בכל ניסוי, ימחקו על מנת להפחית במידע הנשמר.

מודל התקיפה

רעיון והערות מימוש

במטלה זו נתבקשנו לממש שני סוגי מתקפות נפוצות:

- מתקפת $Brute - Force$ – הרצה ממצה של כל מרחב המפתחות עד הצלחת הפריצה (או מעבר גבול זמן/ מספר ניסויים).
- מתקפת $Password - Spraying$ – הרצה של מספר קטן של סיסמאות נפוצות (התקפת מילון) עבור כמה משתמשים נבחרים במקביל, בניגוד למתקפה הקודמת בה מתמקדים בחשבון יחיד.

אופי יצירת משתמשי הדמה

כאשר התחלנו לבנות את המערכת מצאנו כי אופי יצירת משתמשי הדמה פשוט או גולמי מידי להבנתנו ביחס להדגמת יכולות המערכת. במילים אחרות, יצירה של מאגר משתמשים הכולל עשרה משתמשים לפי כל רמת קושי (סיסמה) ויצירת סיסמאות קבועות שאינן משתנות בין ניסוי לניסוי והרצה להרצה ואופי יצירתן מבוסס על מילון המכיל סיסמאות נפוצות, הרגיש פשוט מידי ולא מדגמי.

לפיכך, בחרנו לממש את המערכת כך שיצירת כל אחד ממשתני הדמה תהיה בלתי תלוי בהרצות קודמות, באופי הניסוי והחשוב ביותר, **לא תהיה קבועה**, כלומר לא ניצור פעם אחת (באופן ידני) קובץ משתמשי דמה $(users.json)$.

שמות משתמשי הדמה אינם חשובים להתקפה, מפני שיצאנו מנקודת הנחה כי התוקף יודע את שמות כל המשתמשים במערכת. מכאן, יצירת שמות המשתמשים היא פשוטה ומכילה את סיווג הסיסמה_מספר המשתמש.

יצירת הסיסמה מומשה בדרך מורכבת יותר, ניתן לקבוע את אורך הסיסמה המזערי והמרבי לכל רמת חוזק סיסמה, בנוסף לסט התווים ממנו מורכבת כל סיסמה (ניתן לספק ישירות את סט התווים או לסמן בדגלים אילו קבוצות תווים יכללו בסט התווים).
ניתן לראות זאת בקובץ (הצגה חלקית) $config.json$:

```
"password_parameters": {
  "weak": { "min_length": 4, "max_length": 5, "charset": "0123456ab" },
  "medium": { "min_length": 8, "max_length": 12, "include_upper": true, "include_digits": true }
```

כמו שתיארנו בחלק העוסק במשתמשי הדמה (בחלק המבוא), יצירה זו מגדירה מרחב סיסמאות אפשרי לכל רמת חוזק כך שכל רמה חזקה יותר תכיל מרחב סיסמאות גדול (בהרבה) יותר מקודמותיה.

מדוע בחרנו לממש את התקפת ה-$Brute - Force$ בלבד

ראשית, נציין כי לא בחרנו לממש רק התקפה זו כדי "להוריד" מהעומס של המטלה ובעצם להגיש מטלה חלקית ללא התייחסות כלל להתקפת ה-$Password - Spraying$. לא חיפשנו דרכים לחסוך בעבודה ו-"לעגל פינות" או להגיש את ה-"מינימום המקסימאלי" (או את ה-"מקסימום המינימאלי"). כפי שנסביר עכשיו, ההפך הוא הנכון.

כמתואר לעיל, בכל ניסוי, כל מתקפה, למעשה בכל הרצה כלשהי של שרת האימות, המערכת תיצור כשלושים משתמשי דמה. סיסמתם של כל אחד משלושים משתמשי הדמה האלו תיווצר באופן אקראי ובלתי תלוי לשאר המשתמשים/ההרצה/הניסוי. כל סיסמה מעבר להיותה אקראית, לא תכיל דפוסים נפוצים או לא תהא מורכבת מסיסמאות נפוצות/ידועות או כאלו שניתן לפענח בקלות.

באופן הזה, התקפות מילון למיניהן ובפרט התקפת ה-$Password - Spraying$, לא יהיו רלוונטיות, לא יהיו שונות באופן מהותי מההתקפה הממצה הרגילה $(Brute - Force)$. מפני שלא נעשה שימוש במילון, גם לא כזה המכיל מספר סיסמאות גדול לאין שיעור ובטח שלא כזה המכיל מספר סיסמאות

מצומצם, כל ניסיון של התקפה המנסה כך שמקור הסיסמה לקוח מקובץ (מילון), שקול סטטיסטית להתקפה בה הסיסמה אינה ידועה, אינה לקוחה ממילון כלשהו ונוצרת אקראית.

נניח ומימשנו את ההתקפה השנייה (התקפת ה-$Password - Spraying$), ובכל ניסיון (תחום על ידי גבול עליון של ניסיונות) נבחר סיסמה אקראית ממילון המכיל סיסמאות נפוצות ונבצע ניסיון כניסה למערכת לכל אחד מקובץ המשתמשים הנבחרים בהתקפה זו עם הסיסמה הנוכחית. הסיכוי להצלחה שווה לסיכוי שהסיסמה הלקוחה מהמילון תהיה שווה ללפחות אחת מהסיסמאות של המשתמשים המותקפים.

נסמן כך:

$$\alpha := number\ of\ attacked\ users, \beta := number\ of\ passwords\ in\ common\ dictionary$$

מפני שסיסמאות נוצרות באופן אקראי, הסיכוי שסיסמה כלשהי (מהמילון) תהא שווה לסיסמה כלשהי (של אחד המשתמשים) הוא:

$$\frac{1}{n_0}$$

כאשר n_0 מייצג את מרחב הסיסמאות האפשריות לרמת הקושי הנוכחית.

בניגוד ליצירת סיסמאות ממאגר סיסמאות נפוצות (רחב יותר מהמילון, או שהמילון חלקי אליו ומשתנה בכל הרצה), הסיכוי שאחת מ-β הסיסמאות תתאים לאחד מ-α המשתמשים שקול לסיכוי שסיסמה אקראית כלשהי ממרחב הסיסמאות האפשריות תתאים לאותו משתמש. ולפיכך אין הבדל בין שתי המתקפות.

ברור כי בהתקפה השנייה:

- ניתן להימנע או לכל הפחות לדחות במעט את מספר הניסיונות עד כדי חסימת משתמש (מפני שעבור למשל x ניסיונות נבדוק $\alpha \cdot x$ מקרים, בניגוד למקרה אחד כאשר ההתקפה ממוקדת על משתמש אחד בודד.
- ניתן לראות "כיצד *Limiting-Rate* ברמת משתמש אינו מספיק ללא מגבלות גלובליות[*]".

נציין כי במימוש המערכת שלנו, הגנת $rate\ limit$ (יחד עם $captcha$) פועלת ברמת כתובת IP ואילו הגנות $lockout$ ו-$totp$ פועלות ברמת המשתמש.

לא רצינו להשמיט חלק חשוב זה בבדיקת המערכת ולכן בחרנו לממש את המתקפה הראשונה (והיחידה במימוש שלנו), כך שתדע לקבל דגלים שונים המאפשרים עקיפה/התמודדות מול מנגנוני ההגנה. נתבונן בקובץ $experiments.json$, בחלק ה-$attack_options$ ונבחין בדגלים הבאים:

- דגל $alternate_strength_enabled$ – כאשר דגל זה מורם, בכל ניסיון בהתקפה הסיסמה האקראית שבעזרתה ננסה להיכנס למערכת עם שם המשתמש תשנה על פי רמת קושי אקראית (ללא קשר לחוזק המשתמש המותקף).
- דגל $ip_switch_enabled$ – כאשר דגל זה מורם, בכל שלב בהתקפה בו (בהנחה והגנת $rate\ limit$ ו/או $captcha$ פעילות) תשובת השרת תהא חסימה על פי שם משתמש או הגעה למגבלת הניסיונות המותרים עבור כתובת ה-IP הנוכחית, תוחלף כתובת ה-IP לאחת אחרת אקראית וכך יהיה ניתן להתגבר/לעקוף מנגנון הגנה זה (בדומה לעקרון המרכזי אותו רוצים שנראה עבור התקפת $Password - Spraying$).

[*] הוראות המטלה, עמוד שלישי, תחת הנושא: "$Password - Spraying$".

- דגל $lockout_stop_enabled$ – כאשר דגל זה מורם, בעת החזרת השרת כי המשתמש/כתובת ה-IP חסומה ומעתה והלאה כל ניסיון התחברות (גם ניסיון חוקי) יפסל על ידי המערכת (מפני החסימה), אזי ההתקפה תעצור. לא יהיה טעם להמשיך במתקפה מפני שכל ניסיון נוסף מעבר לנקודה זו יפסל לפני השוואת הסיסמאות.
- דגל $captcha_token_enabled$ – כאשר דגל זה מורם, במידה וההגנת ה-$captcha$ פועלת והמערכת דורשת הזנת קוד $captcha$ זמני, התוקף יספק קוד חוקי וההתקפה תמשך. כמובן שכאשר הדגל אינו מורם, לא יספק התוקף קוד חוקי.

בנוסף בחלק זה מוגדרים ערכים שונים עבור מספר הניסיונות המותרים וזמן השהייה (זמן ההשהיה ברירת המחדל הוא 0), אלו רלוונטיים להרחבות העתידיות ולא נשנה ערכם במדגם הניסויים הנוכחי. שדה חוזק הסיסמה ($password_strength$) קובע את חוזק הסיסמא עבורה נחזיק שם משתמש אקראי מחוזק זה עבור ההתקפה.

על פי הדגלים שהוספנו והאפשרויות השונות לביצוע ההתקפה שמתאפשרות מקיום דגלים אלו, מימשנו את העקרונות התקפת $Password - Spraying$ מנסה להדגים. ולכן נרצה לציין כי בשלב זה אין צורך לממש את המתקפה הזו, או נכון יותר להגיד שמתקפה זו מוכלת (חוץ מהמעבר על כמה משתמשים בו זמנית) בהתקפה שמימשנו.

נובע מכך שלא רק שאין צורך במימוש מתקפה המבוססת על מילון, כל מתקפה אחרת בה מנסים לתקוף כמה משתמשים בכל ניסיון לא תהיה שונה מעקרונות המתקפה הנוכחית ולכן הרצה של הניסיון על α משתמשים אקראיים תהיה שקולה להרצת הניסיון α פעמים (פעם אחת לכל אחד מ-α המשתמשים).

מימוש ההתקפה

קריאה למחלקת ההתקפה ($BruteForceAttack$), תכלול שני פרמטרים:

- לקוח שרת האימות ($client$) – דרכו יבצע התוקף בקשות התחברות (יחד עם בקשות $captcha$ או $totp$ בהתאם להגנות המופעלות).
- חוקי המתקפה ($rules$) – מכילים את כל השדות הרלוונטיים למתקפה:
  - שם המשתמש המותקף.
  - מספר הניסיונות המרבי.
  - מצב הדגלים.
  - ערך ה-$SEED$ (על מנת ליצור קודי $captcha$ זמניים במידה ודגל ה-$captcha_token_enabled$ מורם וההגנה המתאימה פועלת).
  - זמן השהייה (בעת חסימה זמנית/מגבלת ניסיונות, מאותחל ל-0 ולא משתנה עבור מדגם הניסויים הנוכחי).

התחלת ההתקפה תתבצע על ידי קריאה לפונקציית $attack$, ובכל ניסיון (עד הגעה למספר הניסיונות המותרים המרבי או עד עצירת המערכת בהתאם לתגובת השרת) יבוצע ניסיון התחברות. תוך כדי פעילות בהתאם ללוגיקה הנקבעת על פי הדגלים המורמים בעת התנהגות השרת בהתאם למנגנוני ההגנה הפועלים.

לאחר סיום הרצתה יוחזרו:

- סטטוס המתקפה – האם הצליחה או נכשלה.
- מספר הניסיונות שבוצעו עד הפסקת ההתקפה.
- פירוט סיבת סיום ההתקפה.

ניסויים

מהלך הניסויים

נתאר (במבט על, מצומצם כמה שניתן) את שלבי הפעולה הנעשים בעת הרצת הניסויים:

1. יצירת מופע של מחלקת $ExperimentRunner$ האחראית על ריצת הניסויים, עם ציון תיקיית הפלט.
   1.1. הגדרת תתי התיקיות עבור הפלטים השונים, שם קובץ הלוג הכללי.
   1.2. יצירת מופע של מחלקת $ExperimentsGenerator$ האחראית על יצירת כלל הניסויים.
      1.2.1. קריאת קובץ הקונפיגורציה ($experiments.json$).
      1.2.2. יצירת הניסויים.
      1.2.3. חלוקת הניסויים לקבוצות.
   1.3. המשך אתחול המופע (אתחול מופע מחלקת הסטטיסטיקות, שמירת מידע ממופע יצירת הניסויים).
2. הרצת כלל הניסויים על גבי מופע המחלקה (על ידי: `run_all_tests()`).
   2.1. יצירת מופע של מחלקת $ExperimentHandler$ האחראית להרצת הניסויים, עם הפרמטרים (כתובת תיקיית הפלט, שם הקבוצה ורשימת הניסויים).
      2.1.1. יצירת קובץ לוג עבור הניסויים.
      2.1.2. יצירת מופע של מחלקת הלוגים ($Logger$), על מנת לרשום ללוג את תוצאות הניסויים.
   2.2. הרצת פונקציית ריצת הניסויים (על ידי: `run()`).
      2.2.1. עבור כל ניסוי:
         2.2.1.1. יצירת תיקייה זמנית לטובת שמירת הקבצים הנוצרים בעת הקמת שרת האימות.
         2.2.1.2. אתחול השרת על פי ההגנות המוגדרות בניסוי.
         2.2.1.3. אתמול מופע מחלקת התקיפה עם ההגדרות והדגלים כמו שמתואר קודם לכן במסמך.
         2.2.1.4. רשימת זמן התחלת המתקפה (לצורך רישום בלוג).
         2.2.1.5. ביצוע המתקפה.
         2.2.1.6. רשימת זמן סיום המתקפה.
         2.2.1.7. סגירת שרת האימות, העתקת הקבצים (אופציונאלי) לתיקייה שאינה זמנית.
         2.2.1.8. רישום לוג תוצאות מהלך הניסוי.
      2.2.2. הדפסת הלוג הכללי (לטובת בדיקות, אופציונאלי).
   2.3. יצירת גרף זמן הריצה הממוצע לכל ניסיון (על ידי: `plot_all_graphs()`).

רישום המידע

נתאר את חמשת הקבצים הנוצרים במערכת:

קובץ $database.db$

מכיל טבלת משתמשים ($users$) עם ערך שם המשתמש ($username$) והסיסמה המגובבת ($hashed_password$).

קובץ $users.json$

פירוט קובץ זה מובא בחלק קודם יותר של מסמך זה.

קובץ $register.log$

עבור כל רישום משתמש למערכת, תתוסף השורה המכילה את השדות הבאים:

- חותמת הזמן.
- ערך ה-$SEED$.
- כתובת ה-IP.
- שם המשתמש.
- שיטת הגיבוב.
- פרמטרי שיטת הגיבוב.
- ההגנות הפעילות.
- תוצאת השרת.
- סטטוס הבקשה ($HTTP$).
- הפעולה שהתבצעה.
- הודעת תגובת השרת.
- ערך ה-$latency$ של הבקשה.

למשל:


```
{
 "timestamp": "2025-12-28T10:56:34.269498Z",
 "group_seed": "25977022",
 "IP address": "127.0.0.1",
 "username": "alice",
 "hash_mode": "sha256",
 "hash_parameters": {
  "salt_length": 16
 },
 "protection_flags": {
  "pepper": true
 },
 "result": "success",
 "status": 201,
 "action": "register",
 "message": "user created",
 "latency_ms": 18.983
}
```


קובץ $attempts.log$

באופן זהה עבור הקובץ הקודם (ההבדל היחיד הוא כמובן המידע הנרשם), כל ניסיון כניסת משתמש למערכת, תתוסף השורה המכילה את השדות לעיל.

למשל:


```
{
 "timestamp": "2025-12-28T10:56:40.718631Z",
 "group_seed": "25977022",
 "IP address": "127.0.0.1",
 "username": "alice",
 "hash_mode": "sha256",
 "hash_parameters": {"salt_length": 16},
 "protection_flags": {"pepper": true},
 "result": "success",
 "status": 200,
 "action": "login",
 "message": "login success",
 "latency_ms": 1.385
}
```

קובץ $experiments.log$

עבור כל רישום ניסוי התקפת המערכת, תתוסף השורה המכילה את השדות הבאים:

- חותמת הזמן.
- מספר הניסוי.
- פרמטרי ההתקפה.
- שיטת הגיבוב.
- פרמטרי שיטת הגיבוב.
- ההגנות הפעילות.
- תוצאת המתקפה.
- מספר הניסיונות עד עצירה.
- הודעת תוצאת ההתקפה.
- ערך ה-$latency$ של ההתקפה.

למשל:


```
{
  "timestamp": "2025-12-25T11:46:13.860125Z",
  "experiment_id": "0001.log",
  "attack_parameters": {
    "password_strength": "weak",
    "max_attempts": 50000,
    "delay_ss": 0,
    "alternate_strength_enabled": false,
    "ip_switch_enabled": false,
    "lockout_stop_enabled": false,
    "captcha_token_enabled": false,
    "seed": "25977022",
    "username": "weak_user_0"
  },
  "hash_mode": "sha256",
  "hash_parameters": {"salt_length": 16},
  "protections": {},
  "success": true,
  "attempts": 23013,
  "message": "login success",
  "latency_ms": 34830.369
}
```


יצירת מדגם הניסויים

במסמך זה נציג את הרצת הניסויים על פי מדגם מצומצם, ניתן להריץ מרחב מדגם ניסויים רחב בהרבה כמו שצוין קודם לכן ויפורט במעט בהמשך המסמך.

בקובץ `experiments_generator.py`, מוגדרת הלוגיקה ליצירת מדגם הניסוי. כאשר ניגשנו למימוש מנגנון זה, חשבנו לעצמנו מה נרצה לבחון במערכת, אילו שילובי הגנות מעניינות אותנו, איך ניתן לבחון את המערכת בצורה הטובה ביותר אך שלא תדרוש מאיתנו ימי הרצות מלאים, אלא ניתנת להרצה כוללת במספר שעות בודדות.

כפי שציינו בחלק העוסק בשיטות הגיבוב השונות בהן המערכת תומכת, לא כל השיטות זהות בזמן פעילותן הממוצע ויש הבחנה ברורה בין אילו שיטות נחשבות "מהירות" ואילו נחשבות "איטיות". התחשבנו במידע זה ונמנעו מלכתחילה להוסיף מקרי בדיקה עבור בהם שיטת הגיבוב איטית ומספר הניסיונות הצפוי שיבוצעו במתקפה גבוה, מה שיגרום לזמן ריצה ארוך ביותר. חיפשנו איזון בין הדגמה של כלל ההגנות ושיטות הגיבוב השונות אך מניעת הוספה של מקרי בדיקה ארוכים במיוחד שלא מוסיפים ערך רב לניתוח המתקפה.

נציין כי השתמשנו בערך $pepper$ בכל אחד מהניסויים במדגם, בהמשך נפרט מעט על השפעת מנגנון זה על זמני הריצה, אך באופן כללי לחלק זה ניתן להתעלם מחשיבות המנגנון כגורם מעכב זמן ריצה.

ללא הגנות פעילות

Test Number	Hash Mode	Strength	Rate Limit	Lockout	Captcha	Totp	Alternate Strength	IP Switch	Lockout Stop	Captcha Token
1	sha-256	weak	X	X	X	X	X	X	X	X
2	sha-256	medium	X	X	X	X	X	X	X	X
3	sha-256	strong	X	X	X	X	X	X	X	X
4	sha-256	weak	X	X	X	X	V	X	X	X
5	bcrypt	weak	X	X	X	X	X	X	X	X
6	argon2id	weak	X	X	X	X	X	X	X	X

הגנת *rate limit* פעילה

Test Number	Hash Mode	Strength	Rate Limit	Lockout	Captcha	Totp	Alternate Strength	IP Switch	Lockout Stop	Captcha Token
7	sha-256	weak	V	X	X	X	X	V	X	X
8	sha-256	weak	V	X	X	X	X	X	X	X

הגנת *lockout* פעילה

Test Number	Hash Mode	Strength	Rate Limit	Lockout	Captcha	Totp	Alternate Strength	IP Switch	Lockout Stop	Captcha Token
9	bcrypt	strong	X	V	X	X	X	X	V	X
10	bcrypt	strong	X	V	X	X	X	X	X	X

הגנת *captcha* פעילה

Test Number	Hash Mode	Strength	Rate Limit	Lockout	Captcha	Totp	Alternate Strength	IP Switch	Lockout Stop	Captcha Token
11	argon2id	weak	X	X	V	X	X	X	V	X
12	argon2id	weak	X	X	V	X	X	X	X	X
13	sha-256	weak	X	X	V	X	X	X	V	V
14	sha-256	weak	X	X	V	X	X	V	V	X

הגנת $totp$ פעילה

Test Number	Hash Mode	Strength	Rate Limit	Lockout	Captcha	Totp	Alternate Strength	IP Switch	Lockout Stop	Captcha Token
15	sha-256	weak	X	X	X	V	X	X	V	X
16	sha-256	medium	X	X	X	V	X	X	V	X

הגנות $totp$ ו-$captcha$ פעילות

Test Number	Hash Mode	Strength	Rate Limit	Lockout	Captcha	Totp	Alternate Strength	IP Switch	Lockout Stop	Captcha Token
17	sha-256	weak	X	X	V	V	X	X	V	V
18	sha-256	weak	X	X	V	V	V	X	V	V

הגנות $totp$ ו-$lockout$ פעילות

Test Number	Hash Mode	Strength	Rate Limit	Lockout	Captcha	Totp	Alternate Strength	IP Switch	Lockout Stop	Captcha Token
19	sha-256	weak	X	V	X	V	X	V	X	X
20	sha-256	strong	X	V	X	V	X	V	V	X
21	sha-256	strong	X	V	X	V	X	V	V	X

הגנות $rate\ limit$ ו-$captcha$ פעילות

Test Number	Hash Mode	Strength	Rate Limit	Lockout	Captcha	Totp	Alternate Strength	IP Switch	Lockout Stop	Captcha Token
22	sha-256	weak	V	X	V	X	X	V	V	V
23	sha-256	weak	V	X	V	X	X	X	V	V
24	bcrypt	medium	V	X	V	X	X	X	X	X

כלל ההגנות פעילות

Test Number	Hash Mode	Strength	Rate Limit	Lockout	Captcha	Totp	Alternate Strength	IP Switch	Lockout Stop	Captcha Token
25	sha-256	weak	V	V	V	V	X	V	V	V
26	sha-256	weak	V	V	V	V	X	V	X	V
27	argon2id	medium	V	V	V	V	V	V	V	X

מרחב המדגם המלא

Test Number	Hash Mode	Strength	Rate Limit	Lockout	Captcha	Totp	Alternate Strength	IP Switch	Lockout Stop	Captcha Token
1	sha-256	weak	X	X	X	X	X	X	X	X
2	sha-256	medium	X	X	X	X	X	X	X	X
3	sha-256	strong	X	X	X	X	X	X	X	X
4	sha-256	weak	X	X	X	X	V	X	X	X
5	bcrypt	weak	X	X	X	X	X	X	X	X
6	argon2id	weak	X	X	X	X	X	X	X	X
7	sha-256	weak	V	X	X	X	X	V	X	X
8	sha-256	weak	V	X	X	X	X	X	X	X
9	bcrypt	strong	X	V	X	X	X	X	V	X
10	bcrypt	strong	X	V	X	X	X	X	X	X
11	argon2id	weak	X	X	V	X	X	X	V	X
12	argon2id	weak	X	X	V	X	X	X	X	X
13	sha-256	weak	X	X	V	X	X	X	V	V
14	sha-256	weak	X	X	V	X	X	V	V	X
15	sha-256	weak	X	X	X	V	X	X	V	X
16	sha-256	medium	X	X	X	V	X	X	V	X
17	sha-256	weak	X	X	V	V	X	X	V	V
18	sha-256	weak	X	X	V	V	V	X	V	V
19	sha-256	weak	X	V	X	V	X	V	X	X
20	sha-256	strong	X	V	X	V	X	V	V	X
21	sha-256	strong	X	V	X	V	X	V	V	X
22	sha-256	weak	V	X	V	X	X	V	V	V
23	sha-256	weak	V	X	V	X	X	X	V	V
24	bcrypt	medium	V	X	V	X	X	X	X	X
25	sha-256	weak	V	V	V	V	X	V	V	V
26	sha-256	weak	V	V	V	V	X	V	X	V
27	argon2id	medium	V	V	V	V	V	V	V	X

תוצאות

אחרי ריצה מוצלחת של 27 הניסויים המוגדרים במרחב הניסויים ניצור באופן אוטומטי שלושה גרפים. בחלק זה של המסמך נציג את הגרפים האלו ובפרק הבא העוסק בניתוח נבצע ניתוח מקיף של המידע והנקודות החשובות העולות מהם.

הרצת כל ניסוי איננה עקבית ותוצאה המתקבל בהרצת ניסוי כלשהו לא מובטחת שתתקבל שוב ושוב עבור כל הרצה של ניסוי זה, יש לזכור שיצירת הסיסמאות (למשתני הדמה ובכל ניסיון בהתקפה) הינו אקראי ורצוי שנלמד מכל גרף את המגמות הבולטות ולא נתעכב על ערכים מסוימים, שכן לא מובטחת עקביות ערכית, אבל כן מובטחת עקביות מגמתית.

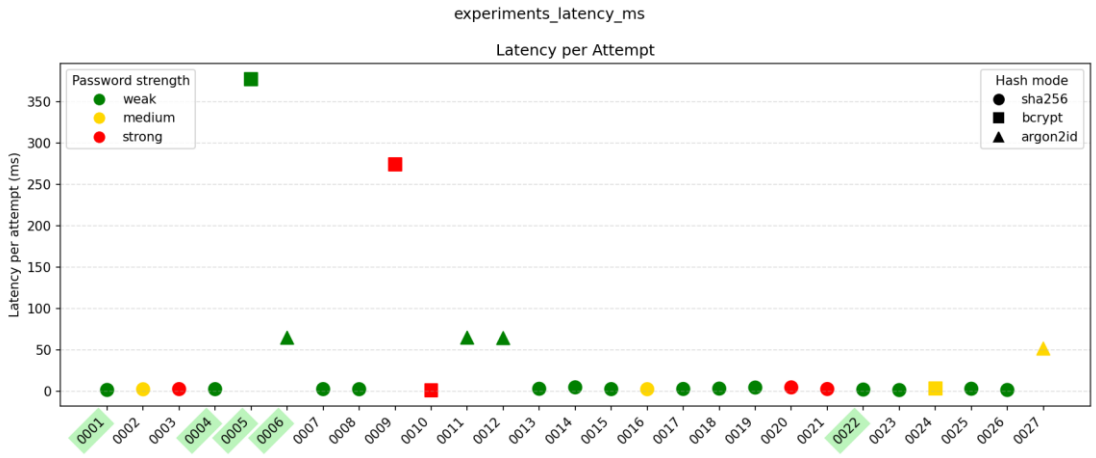
גרף המתאר את זמן ה-$latency$ הממוצע בכל ניסוי

בתום ההרצה הכוללת עבור כל ניסוי תהיה שורה מתאימה בקובץ הלוג הכללי ($experiments.log$). על מנת לחשב את זמן ה-$latency$ הממוצע בכל ניסוי ניתן לבצע חישוב פשוט:

$$\frac{Experiment\ latency}{Number\ of\ attempts}$$

בכדי להפוך את הגרף לכמה שיותר קריא, כל שיטת גיבוב מאופיינת בצורה אחרת ובאופן דומה כל חוזק סיסמה צבוע בצבע אחר. יתרה מכך, על מנת לבדל את הניסויים בהם ההתקפה הצליחה, תווית שם הניסוי עבור כל ניסוי שכזה תודגש בצבע כחול.

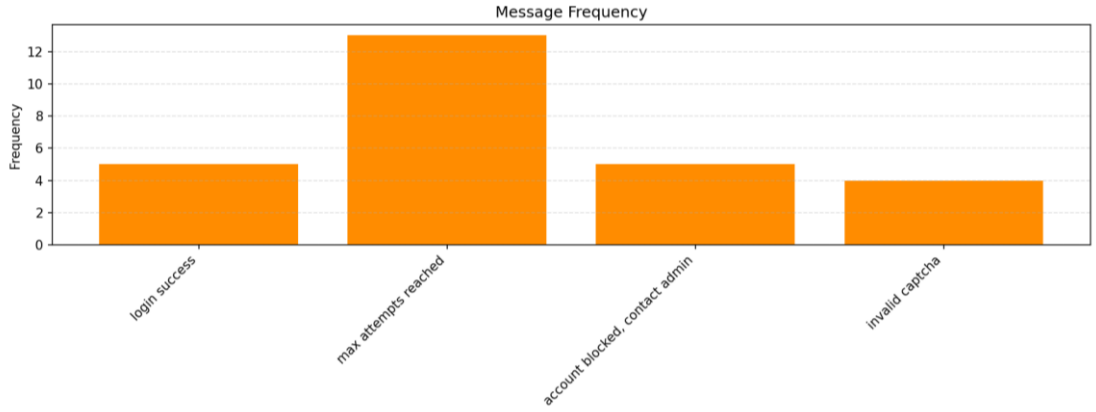
הגרף המתקבל (עבור הרצת מדגם הניסויים שלעיל):



גרף המתאר את שכיחות כל הודעת סיבת סיום ההתקפה

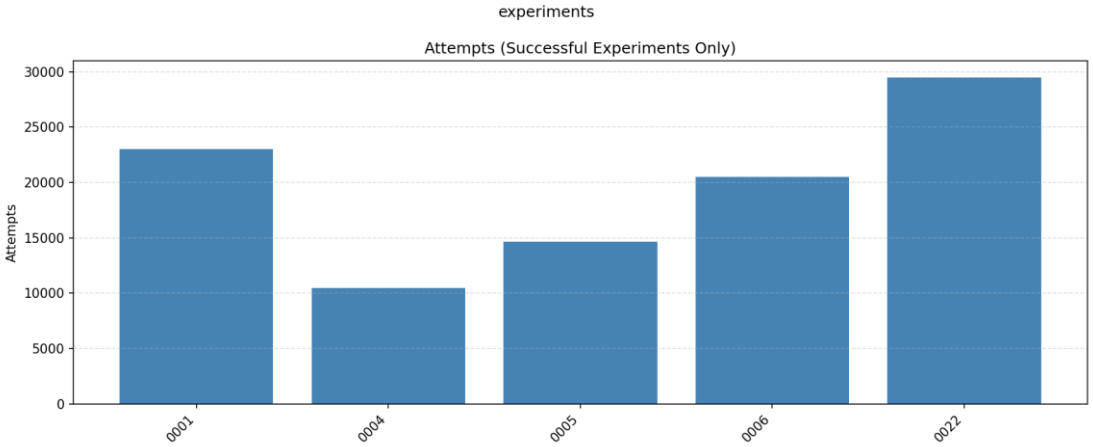
בשביל שבשלב הבא נהיה יכולים לקבוע את השפעות ההגנות, חוזקן במערכת ויכולת התמודדותן לזהות, לעצור ולהתנגד להתקפות, חשוב שנשים לב לאופי תוצאות הניסויים.

להלן גרף המתאר את שכיחות הודעות החזרה מהמתקפה, כל הודעה שכזו מתאר את הסיבה הגרמה לעצירת המתקפה/סיומה אחרי הצלחה או הגעה לגבול עליון של ניסיונות כניסה:



גרף המתאר את מספר הניסיונות עבור כל ניסוי בו ההתקפה הצליחה

נקודה נוספת שחשבנו שכדאי להמחיש בצורה גראפית ולא רק לתאר על פי תוצאות קובץ הלוג הכללי, היא מספר הניסיונות שנדרשו עבור כל התקפה מוצלחת. כמו שנתאר בהמשך, ההתקפות המוצלחות אירעו רק עבור שם משתמש בעל סיסמה חלשה והגרף הבא מציג את מספר הניסיונות האלו (נזכיר שמדגם הסיסמאות של סיסמה חלשה מכיל כ-65610 סיסמאות חוקיות):



ניתוח

על פי התוצאות שהוצגו בפרק הקודם, ניתן להסיק מידע רב ומעניין לגבי פעולת המערכת.

שיטת הגיבוב

נתבונן בגרף המתאר את זמן ה-$latency$ הממוצע בכל ניסוי, כעת עולות המסקנות הבאות לגבי שיטת הגיבוב:

- זמן ה-$latency$ הממוצע עבור כל הניסויים בהם נעשה שימוש בשיטת הגיבוב $sha-256$ דומה ועומד על ערכים סביב נקודת ה-0. הסתכלות בקובץ הלוג מספרת לנו כי הערך הממוצע נע בין 1.1 ל-1.8 מילישניות כך שברוב המקרים הערך הממוצע קרוב ל-1.35.
- זמן ה-$latency$ הממוצע עבור הניסויים בהם נעשה שימוש בשיטת הגיבוב $bcrypt$ עומד על כ-300 מילישניות. פי יותר מ-220 מהזמן הממוצע בשיטת הגיבוב $sha-256$.
  ערך זה נחשב מהיר ביותר וניתן להשתמש באינטרפולציה כדי לחשב את הזמן הדרוש על
- זמן ה-$latency$ הממוצע עבור הניסויים בהם נעשה שימוש בשיטת הגיבוב $argon2id$ עומד על כ-75 מילישניות. מהיר יותר מהשיטה הקודמת אך איטי בהרבה מהשיטה הראשונה.

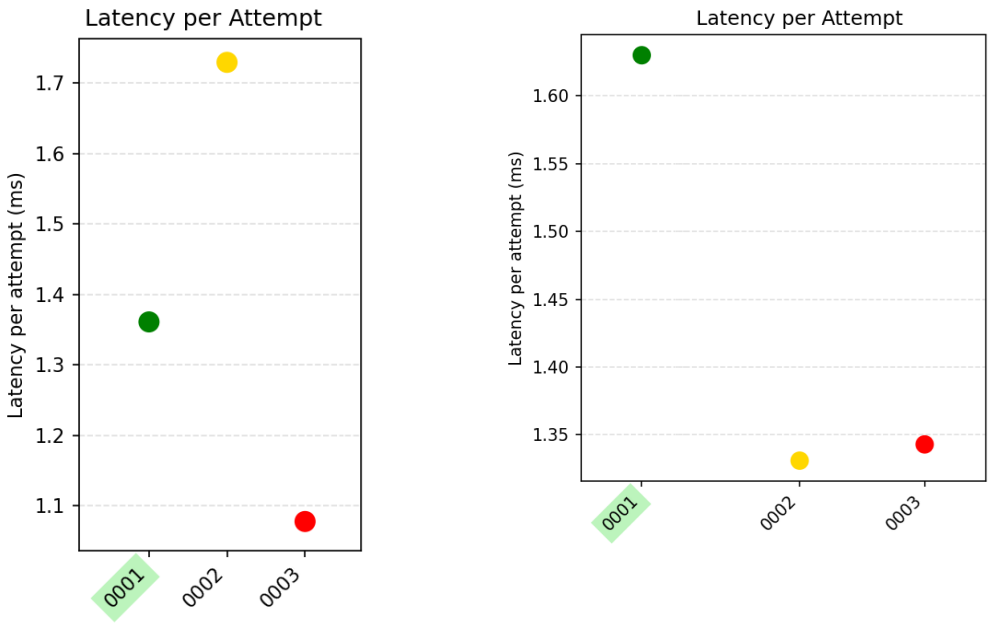
בנוסף, ניתן להשתמש באינטרפולציה על מנת לשער את הזמן הממוצע לפענוח סיסמה עבור כל אחת משיטות הגיבוב (על ידי הרצה ממצה על מרחב הסיסמאות).

Hash Mode/ Strength	Weak	Medium	Strong
Sha-256	1.4762 minutes	144,595,636 years	136000000000000 years
Bcrypt	5.4675 hours	32132365001 years	3023400000000000 years
Argon2id	1.366875 hours	8033090920 years	7558500000000000 years

השפעת ה-$pepper$

נרצה לבחון האם ישנה השפעה כלשהי להוספת ערך $pepper$ המתואר בחלקים קודמים של המסמך על זמן ה-$latency$ הממוצע. חלק מהניסויים נעצרו אחרי חסימה קבועה מהשרת וכפי שנתאר בהמשך, חסימה זו התקבלה אחרי בין 3 ל-5 ניסיונות.

לפיכך, נתמקד רק ב-3 הניסויים הראשונים ($sha-256$ ללא הגנות וחוזק סיסמה חלש, בינוני וחזק).
מצד ימין מופיע קטע הגרף הרלוונטי עבור 3 הניסויים האלו כאשר אין שימוש ב-$pepper$ ומצד שמאל כאשר יש שימוש ב-$pepper$ (השתמשנו בערך 51514848):

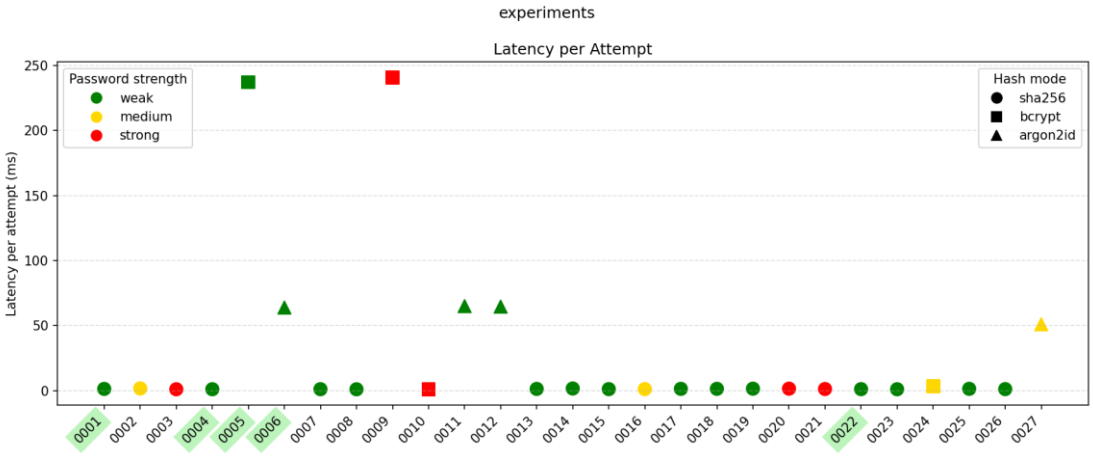


נשים בצד את ערכי זמן ה-$latency$ הממוצע (מהסיבות שלעיל) ונתמקד דווקא במגמות העולות משני הגרפים החלקיים. שיטת הגיבוב $sha-256$ מקבלת קלט בכפולות של בלוקים בגודל קבוע, ככל שהקלט דורש ריפוד להוספת ביטים כך שיתאים למספר שלם של בלוקים, זמן החישוב יתארך.

במקרה שמובא למעלה, ניתן לשער כי דווקא עבור סיסמאות קצרות עקב פעולת הריפוד הארוכה יותר, זמן החישוב יגדל ביחס לסיסמאות ארוכות יותר שדורשות פחות ריפוד. תוספת ערך ה-$pepper$ קצרה יחסית ולכן ההשערה נכונה גם ללא התייחסות ל-$pepper$.

באופן כללי, עבור שיטת הגיבוב $sha-256$, זמן החישוב עם או בלי $pepper$ הוא קצר מאוד וכאשר מדובר בערכים סביב ה-1 מילישנייה, יכולת המדידה האמיתית לא אמינה במיוחד וההשפעה של ה-$pepper$ קטנה עד בלתי מורגשת.

כאשר נתייחס למנגנון ה-$pepper$ ביחס שימוש בשתי שיטות הגיבוב האחרות עבורן זמן ה-$latency$ הממוצע גבוה בקנה מידה משמעותית מזה של שיטת הגיבוב $sha-256$. במקרה זה ההשפעה זניחה לחלוטין, כפי שניתן לראות בגרף הבא המציג את זמן ה-$latency$ הממוצע עבור מדגם הניסויים הנוכחי כך שבכל ניסוי ישנו שימוש ב-$pepper$:



חוזק הסיסמה

מהגרף המתאים בפרק הקודם ניתן לראות כי לחוזק הסיסמה אין כמעט השפעה משמעותית על זמן ה-$latency$ הממוצע **אלא** אם ישנה חשיבות אמיתית להבדלים סביב זמן ממוצע מהיר ביותר באופן כללי.

באופן דומה לנקודה למסקנה הקודמת, אם נתבונן בגרף החלקי (רק ב-3 הניסויים הראשונים) נסיק כי גורם ההשפעה העיקרי בשוני זמן הריצה במקרים אלו, הוא מידת הריפוד הנדרשת לכל סיסמה עבור שיטת הגיבוב $sha-256$. סיסמאות חלשות שאורכן קצר, ידרשו ריפוד נוסף דבר שמעלה את הזמן הדרוש לחישוב.

ביחס לשיטות הגיבוב השונות שפעולתן שונה מזו של השיטה הראשונה נסיק כי אין כמעט הבדל בזמן ה-$latency$ עבור רמות חוזק שונות, וסטייה של ± 3 מילישניות כאשר הזמן הממוצע הוא 75 או 300 מילישניות היא זניחה ויכולה לקרות גם מגורמים אחרים במערכת.

נסיק מכך שרק בקנה מידה בו שיטת הגיבוב איתה נעבוד מהירה מאוד ויש חשיבות עמוקה לסטיות של מספר מילישניות בודדות, רק אז ישנה השפעה כלשהי שאינה בהכרח זניחה בזמן ה-$latency$ עבור אורך סיסמה **קצר** יותר.

ההבדל בזמן ה-$latency$ בין הניסוי הראשון לשלישי הוא של כ-60% (ובאופן דומה בין שני הניסויים בעת שימוש ב-$pepper$, כ-40%). עבור המהירות המתקבלת לא נרשמת מגמה כה משמעותית, אך בו בעת לא ניתן להתעלם ממסקנה זו.

נקודה חשובה נוספת, התבוננות בגרף המייצג את מספר הניסיונות עבור כל התקפה מוצלחת וחישוב ממוצע באופן ידני מגלה לנו כי במדובר בממוצע ב-19625 ניסיונות. פונקציית יצירת הסיסמאות בה משתמש התוקף בכל ניסיון התקפה מחזירה סיסמה אקראית מבין מרחב הסיסמאות האפשרי. נבחין כי בכל אחת מההתקפות המוצלחות נעשה המשתמש המותקף היה בעל סיסמה חלשה.

מרחב הסיסמאות החלשות הוא כאמור 65610 סיסמאות חוקיות ועל פניו גדול בכשלושים אחוזים ממספר הניסיונות המרבי בכל ניסוי (50000), אך מפני שיצירת הסיסמאות בכל שלב בניסוי אינה מתנהלת באופן ממצה כלומר אין מעבר סידורי על כל המרחב ודווקא מדובר בפונקציה אקראית, אפשר להתרשם כי מספר הניסיונות הממוצע הוא כשלושים אחוז בלבד מגודל מרחב הסיסמאות החלשות.

נובע מכך שאפילו שפונקציית יצירת הסיסמאות הינה אקראית ולכן באופן תאורטי קיימת אפשרות בה לכל ערך מרבי של מספר ניסיונות לא יצליח התוקף במתקפה שלו, באופן ממוצע כמעט תמיד מובטחת התקפה מוצלחת כאשר מספר הניסיונות המרבי הוא לפחות 30% ממרחב הסיסמאות החלשות. דבר שרק מעיד על הכשל החמור ביצירת סיסמאות חלשות באופן הנוכחי.

גורם עצירת המתקפה

נתבונן בגרף המתאר את שכיחות כל הודעת סיבת סיום ההתקפה, גרף זה אינו גדוש במידע והוא מספק לנו תובנות בודדות אך חשובות מאוד.

- מבין 27 הניסויים שהרצנו, 5 התקפות הצליחו. כאשר "נצלול" אל קובץ הלוג מיד נראה כי כל ההתקפות המוצלחות התבצעו כנגד סיסמה חלשה (מרחב סיסמאות קטן מאוד) ולמרות שבכל ניסיון הסיסמה נוצרת באופן אקראי ואין מעבר ממצה על כלל מרחב הסיסמאות האפשרי וכתוצאה מכך לא מובטחת הצלחה, עדיין במסגרת 50000 הניסיונות, הצלחנו למצוא התאמה.

- כ-13 ניסויים הסתיימו עקב הגעה למספר ניסיונות מרבי, מה שמעיד שלמרות ההרצות ללא הגנות כלל וההרצות בהן דגלים מסוימים העוזרים להתמודדות עם מנגנוני הגנה מסוימים היו מורמים, רוב הניסויים לא צלחו (כולל יתר הניסויים הבאים).

- כ-5 ניסויים הסתיימו עקב חסימת משתמש/חסימת כתובת IP וניתן לקבוע כי מנגנון זה פועל בצורה טובה. אם נסתכל בקובץ הלוג נבחין כי עבור כל ניסוי בו לא פועל מנגנון הגנה מגביל (המבצע חסימה של משתמש ו/או כתובת IP), ולא מורמים דגלים להחלפת כתובת ה-IP או יצירת קוד $captcha$ חוקי, בכל המקרים האלו אחרי בין 3 ל-5 ניסיונות (בהתאמה לאילו הגנות פעלו ושילובן) נחסם המשתמש ו/או כתובת ה-IP ולמעשה המערכת הצליחה בזיהוי המתקפה ועצרה אותה במהירות, מה שלא רק חוסך בזמן המתקפה אלא גם חוסך משאבים לשרת האימות (הוא פחות "תפוס").

- כ-4 ניסויים הסתיימו עקב שגיאת קוד $captcha$. בדומה לנקודה הקודמת, תוצאה זו מדגימה את יכולתה של המערכת לחסום משתמשים על בסיס קוד $captcha$ שגוי (דגל יצירת קוד $captcha$ החוקי על ידי מעקף מנהל לא היה מורם).

לא ניתן להתעלם מהצלחתה של המערכת לזהות, להתמודד ולעצור התקפות בזמן. נכון, ישנן נקודות חולשה של המערכת (בעיקר מרחב הסיסמאות החלשות הקטן ביותר ו/או שימוש בשיטת גיבוב מהירה כגון $sha-256$). אך, כאשר ההגנות הנכונות מופעלות ואין עקיפה של מנגנוני ההגנה, אפשר לקבוע כמעט בסבירות מוחלטת כי כל התקפה תיכשל, בין אם מחסימה ובין אם מהגעה למספר הניסיונות המותר.

מסקנות חשובות

נציג את הנקודות החשובות ביותר שהסקנו מתוצאות הניסויים, מהבדיקות שעשינו בכל שלבי בניית המערכת ומאופי מימוש המערכת ומנגנוני ההגנה.

- שימוש בסיסמאות חזקות – ראינו ואף הצגנו אינטרפולציה של זמני ריצת המערכת הדרושים למעבר ממצה על מרחב הסיסמאות וניתן לקבוע באופן מוחלט כי ככל שאורך הסיסמה גדול יותר וככל שמרחב הסיסמאות האפשריות רחב יותר, הזמן הדרוש בהתקפות אלו הוא אינו זמן סביר. טרם הפעלת מנגנוני הגנה למיניהם, המעבר הממצה על מרחב סיסמאות גדול (למשל עבור הסיסמאות החזקות) פשוט לא ריאלי (במסגרת יכולות חישוב סטנדרטיות) ניתן ללמוד כי המפתח להגנה אמיתית, מעבר להגנות ומנגנוני התמודדות, הוא בחירה נכונה של סיסמאות.

- שיטות גיבוב איטיות – מצאנו כי מבין שלוש שיטות הגיבוב המתוארות מעלה, השיטה האיטית ביותר היא $bcrypt$. שילוב של מרחב סיסמאות רחב והגברת זמן הדרוש לכל ניסיון כפי שמתאפשר על ידי שימוש בשיטת הגיבוב שלעיל, גורמות לזמן ריצה כה ארוך שגם במערכות מחשוב מהירות ומתקדמות לא יהיה ניתן להתמודד עם זאת ביעילות.

  למדנו כי שימוש בשיטות גיבוב מהירות הוא לא כדאי במסגרת שרת האימות וכאשר מדובר בזמני בקשות בודדות אזי למרות שימוש בשיטות גיבוב איטיות, מבחינת המשתמש אין הבדל נראה לעין. אך מבחינת המערכת והתמודדותה נגד התקפות מדובר בהבדל ענק.

- חסימות משתמשים/כתובות IP – מבין כלל ההגנות שממשנו מצאנו כי ההגנות החוסמות שם משתמש או כתובת IP על סמך מעבר מספר ניסיונות כושלים הן ההגנות האפקטיביות ביותר. בממוצע אחרי כארבעה ניסיונות כושלים הצלחנו לחסום את התוקף ולעצור את ההתקפה.

  כמובן אין לזלזל ביכולת התוקף לשנות את כתובת ה-IP שלו או את שם המשתמש תחת המתקפה. ויש להתייחס לנקודות חולשה אלו בכל ניסיון עתידי לחיזוק המערכת.

- הכשל בהגנת $TOTP$ – הגנה זו פועלת רק כאשר התוקף הצליח לנחש נכונה את סיסמת המשתמש ולכן בכל פעם שתכשל ההתקפה על ידי מנגנון זה, ידע התוקף כי הצליח לנחש נכונה את סיסמת המשתמש ויהיה יכול להשתמש בזו על מנת לנסות לפרוץ לחשובות אחרים של המשתמש או להמתין לרגע בו הגנה זו לא תהא פעילה.

הרחבות עתידיות

בחלק זה נעסוק בקצרה בתיאור של הרחבות עתידיות שניתן להוסיף במערכת.

ניסויים

כפי שצוין קודם לכן, ניתן להרחיב את מרחב מדגם הניסויים כך שיכיל כל שילוב אפשרי של:

- חוזק סיסמה.
- דגלי התקפה.
- הגנות פעילות.
- פרמטרי הגנות.
- שיטות גיבוב.

באופן זה יהיה ניתן לבחון את המערכת במלואה, עבור כל תצורה אפשרית וחוקית. מן הסתם הרצה שכזו תדרוש משאבים רבים וזמן ממושך להרצה. עבור דרישות המטלה המוגדרת אין טעם להריץ את מרחב הניסויים הכולל וזאת מטעמי זמן.

הגנות

כדי לחזק את המערכת ניתן להוסיף את ההגנות הבאות:

- השהייה מדורגת של המערכת:
  - חמישה ניסיונות כושלים – ללא השהייה.
  - שישה לעשרה ניסיון כושלים – השהיית המערכת לכמה שניות (אחרי כל ניסיון כושל).
  - מעל לעשרה ניסיונות כושלים – השהיית המערכת לדקה (אחרי כל ניסיון כושל).
- שינוי הגבלת הקצב לא רק על פי כתובת IP, אלא עבור:
  - שם משתמש.
  - שילוב של שם משתמש וכתובת IP.
- זיהוי דפוסים חריגים:
  - ניסיונות כושלים רבים בזמן קצר, על שם משתמש מסוים.
  - ניסיונות רבים מכתובת IP שונות.
  - ניסיונות כניסה למשתמשים חסומים בשינוי כתובת IP.

בנוסף, נהיה יכולים לדרוש מכל המשתמשים יצירת סיסמה חזקה בלבד, המורכבת מלפחות 12 תווים וחייבת להכיל תו אחד לפחות מכל קבוצת תווים. כך מרחב הסיסמאות יהיה כה גדול שהתקפה ממצה תהיה בעלת סיכוי אפסי לניחוש הסיסמה הנכונה.

יתרה מכך, ניתן להוסיף מנגנוני אימות בשני גורמים ($2FA$), להטמיע הגבלות דינאמיות על פי קצת הבקשות מכתובת מסוימת או על שם משתמש מסוים.

מקורות

[1] Question 5, assignment 12 ((ממ"ן) מטלת מנחה), Bar Toplian*.

[2] "Introduction to Video-Based Cryptography" ("מבוא לקריפטואנליזה מבוססת וידאו"), Bar Toplian. https://digitalwhisper.co.il/files/Zines/0xAB/DW171-4-CryptoVideo.pdf.

[3] Wikipedia contributors. (2025, November 8). Argon2. In *Wikipedia, The Free Encyclopedia*. Retrieved from https://en.wikipedia.org/w/index.php?title=Argon2&oldid=1321037446.

* אחד משני השותפים האחראים להגשת מטלה זו.